

JavaScript-11

DOM-optimering

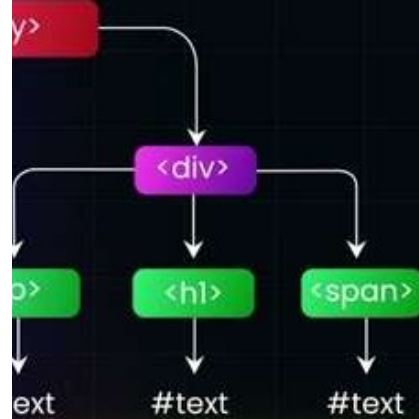
30 september 2026

JavaScript – utan tjafs

Författare: Acke Strömberg



För en kanin



DOM-optimering

När vi bygger små webbapplikationer spelar det inte så stor roll hur vi skriver vår kod. Men när applikationerna växer, och vi börjar hantera hundratals knappar eller tunga sökfält, då måste vi börja tänka på prestanda. Att manipulera och lyssna på HTML-dokumentet (DOM:en) är nämligen det "dyraste" och mest tidskrävande en webbläsare kan göra. I det här kapitlet ska vi lära oss hur vi interagerar med webbläsaren på ett smart sätt.

Event Delegation – spara på webbläsarens minne

Tänk dig att du bygger en att-göra-lista (To-Do app) eller en webbshop där du har en lista med 100 artiklar, och varje artikel har en "Ta bort"-knapp. Det vanliga sättet att lösa detta är att loopa igenom alla knappar och lägga till en händelselyssnare (`addEventListener`) på varenda en. Det betyder 100 unika lyssnare som ligger och drar minne i webbläsaren. Detta blir snabbt ineffektivt. Lösningen kallas Event Delegation. Istället för att sätta en lyssnare på varje enskild knapp, sätter vi en enda lyssnare på hela föräldra-elementet (t.ex. `<ul>`-listan).

```
[ Klicka på <button> ]  
  —→ Bubblar upp till <li>  
  —→ Bubblar upp till <ul id="minLista"> (Här fångar vi det!)
```

Bild 1: Klickhändelsen färdas uppåt som en bubbla genom alla föräldra-element.

Tack vare att händelser i JavaScript "bubblar" uppåt i HTML-trädet, kan vi fånga upp klicket på föräldern och sedan kontrollera exakt vilket element det var användaren faktiskt klickade på.

```
const lista = document.querySelector('#minLista');  
  
// Vi sätter EN ENDA lyssnare på hela listan (föräldern)  
lista.addEventListener('click', (event) => {  
  
  // event.target är det exakta elementet som muspekaren klickade på  
  if (event.target.tagName === 'BUTTON') {  
    console.log(`Klick på knappen inuti: ${event.target.parentElement.textContent}`);  
    event.target.parentElement.remove(); // Tar bort hela listobjektet (li)  
  }  
});
```

Bild 2 Genom att använda `event.target` kan vi filtrera ut klick från specifika element.

Debounce – Stoppa stressade funktioner

Vissa händelser i webbläsaren triggas otroligt ofta. Ett klassiskt exempel är när en användare skriver i ett sökfält (input) och vi vill hämta sökresultat live från ett API, eller när användaren ändrar storlek på fönstret (resize). Om en användare skriver snabbt och söker efter ordet "JavaScript" (10 bokstäver), och vi lyssnar på det vanliga input-eventet, kommer vi att skicka 10 stycken API-anrop på under två sekunder. Det stressar både webbläsaren och vår server helt i onödan. Med Debounce sätter vi en dörrvakt (en timer). Vi säger till JavaScript: "Vänta med att köra funktionen tills användaren har tagit en paus i skrivandet och det har varit tyst i 500 millisekunder".

Användaren skriver:

J — a — v — a — S — c — r — i — p — t ... (Väntar 500ms) ...
→ [API-anrop körs!]

Bild 3: Istället för ett anrop per bokstav väntar systemet tills användaren har skrivit klart.

Se kodexemplet på bild 4 för lite djupare förståelse.

```
function debounce(funktion, fördröjning) {  
  let timer;  
  
  return function(...args) {  
    clearTimeout(timer); // Avbryt den förra timern om användaren trycker på en ny tangent  
    direkt  
    timer = setTimeout(() => {  
      funktion(...args); // Kör funktionen först när tiden har gått ut helt  
    }, fördröjning);  
  };  
}  
  
// Funktionen vi vill köra mer sällan  
const hämtaSökResultat = (event) => {  
  console.log(`Hämtar data för: ${event.target.value}`);  
};  
  
const sökFält = document.querySelector('#sökInput');  
// Vi packar in vår funktion i debounce med en fördröjning på 500ms  
sökFält.addEventListener('input', debounce(hämtaSökResultat, 500));
```

Bild 4: Nu skickas endast ett samlat anrop när användaren har tagit en paus i sitt skrivande.

DocumentFragment – Bygg osynligt innan du visar

Varje gång du använder `appendChild()` för att trycka in ett nytt HTML-element på sidan måste webbläsaren räkna om hur hela designen och layouten ska ritas upp på skärmen (detta kallas för en *reflow*). Om du har en loop som lägger till 50 nya produkter i en lista efter ett API-anrop, och du kör `appendChild()` inuti loopen, tvingar du webbläsaren att göra denna tunga omräkning **50 gånger i rad**. Det kan få sidan att lagga.

```
BRA:
Loop → [ Lägg till i osynlig låda ]
      → [ Tryck ut allt på sidan samtidigt ]
(Sidan ritas om 1 gång)
-----
DÅLIGT:
Loop → [ Lägg till 1 ]
      → [ Lägg till 2 ]
      → [ Lägg till 3 ]
(Sidan ritas om 3 gånger)
```

Bild 5 Genom att samla alla element i ett fragment ritas sidan bara om en enda gång.

Lösningen är att använda ett **DocumentFragment**. Det är som en osynlig låda eller en temporär minnesyta. Du lägger till alla dina 50 element i lådan först (vilket sker blixtnabbt eftersom det sker utanför skärmen), och sedan tömmer allt från lådan på sidan **en enda gång**.

```
const produktLista = document.querySelector('#produkter');

// Vi skapar ett osynligt fragment (en skugglåda)
const fragment = document.createDocumentFragment();

const nyaProdukter = ['Skor', 'Tröja', 'Keps', 'Jacka'];

nyaProdukter.forEach((produktNamn) => {
  const li = document.createElement('li');
  li.textContent = produktNamn;

  // Vi lägger till elementet i fragmentet (inte på den riktiga skärmen än!)
  fragment.appendChild(li);
});

// Nu trycker vi ut hela paketet på skärmen på en och samma gång
produktLista.appendChild(fragment);
```

Bild 6 Webbläsaren behöver bara göra en enda layout-omräkning.